code optimization Contd:

(1) **Common sub expression elimination:**

*. If it was previously computed
*. values of variables have not changed.

Ex:

```
a = b + c
b = a - d
c = b + c
d = a - d
```

⟹

```
a = b + c
b = a - d
c = b + c
d = b
```

(2) **Constant Folding:**

If the value of an Exp^n is constant, use the constant instead of an Exp^n.

```
Pi = 22/7
```

⟹

```
Pi = 3.14
```

(3) **Copy propagation:**

Ex

```
x = a
y = x * b
z = x * c
```

⟹

```
x = a
y = a * b
z = a * c
```

(4) **Dead code Elimination:**

If a variable is live, if its value can be used subsequently, otherwise it is dead.

```
x = a
y = a * b
z = a * c
```

⟹

```
y = a * b
z = a * c
```

5. Code Motion: Moves code outside a loop.

```
while (i<10)
{
    x=y+z;
    i=i+1;
}
```

⟹

```
x=y+z;
while (i<10)
{
    i=i+1;
}
```

6. Induction variable Elimination & Reduction in Strength:

```
i=1;
while (i<10)
{
    t=i*4;
    i=i+1;
}
```

⇒

```
t=4;
while (t<40)
{
    t=t+4;
}
```

Instruction Cost Contd:

* Determine the cost of the following sequence of instructions:

①
```
LD  R0, y        1+1=2
LD  R1, z        1+1=2
ADD R0, R1, R1   1+0=1
ST  x, R0        1+1=2
                 Total Cost = 7
```

②
```
LD  R0, i        1+1=2
MUL R0, R0, 8    1+1=2
LD  R1, a(R0)    1+1=2
ST  b, R1        1+1=2
                 T.C = 8.
```

③
```
LD  R0, c        1+1=2
LD  R1, i        1+1=2
MUL R1, R1, 8    1+1=2
ST  a(R0), R0    1+1=2
                 8
```

④
```
LD  R0, p        1+1=2
LD  R1, 0(R0)    1+1=2
ST  x, R1        1+1=2
                 6
```

⑤
```
LD  R0, x        1+1=2
LD  R1, y        1+1=2
SUB R0, R0, R1   1+0=1
BLTZ *R3, R0     1+1=2
Branch instruction
                 7
```

A Simple Code Generator:

- Generates target code for a sequence of 3-address statements.
- For each operator in a statement, there is a corresponding target language operator.
  +, −, (3AC) ⇒ ADD, SUB (Target lang)

Register & address descriptor

① Register descriptor: keeps track of what is currently in each register. ex R0, R1

* Initially all registers are empty.

② Address descriptor: keeps track of the location where the current value of the name can be found.

* Location may be register, a stack location or memory address.

A code generation algorithm:

For each three address statement of the form

x = y op z,

① Invoke a function getreg (get-register) to determine the location L, where result of y op z should be stored. [Empty register, occupied register or memory loc^n]

② Approach address descriptor for y, to determine y', the current location of y. If y is not already

in L, generate MOV y', L.

(3) Generate the instruction op z', L. update address descriptor of x to indicate that x is in L. If L is a register, update its descriptor to indicate that it contains the value of x.

(4) If y and z have no next uses & not live on exit, update the descriptors to remove y & z.

Ex] $d = (a-b) + (a-c) + (a-c)$

Equi. Three address code sequence is

$t_1 = a-b$

$t_2 = a-c$

$t_3 = t_1 + t_2$

$d = t_3 + t_2$

Statements	Code generated	Register descriptor	Address descriptor
$t_1 = a-b$	MOV a, R0 SUB b, R0	Register are empty R0 contains t_1	t_1 is in R0
$t_2 = a-c$	MOV a, R1 SUB c, R1	R0 contains t_1 R1 —— t_2	t_1 in R0 t_2 in R1
$t_3 = t_1 + t_2$	ADD R1, R0	R0 contains t_3 R1 —— t_2	t_2 in R1 t_3 in R0
$d = t_3 + t_2$	ADD R1, R0 MOV R0, d	R0 contains d	d in R0 d in R0 & Memory.

The DAG representation of Basic BLOCKS:

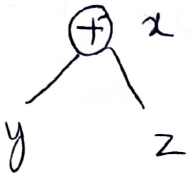
① Determines common subexpressions

② Leaves are labelled by variable names or constants. Initial values are subscripted with 0.

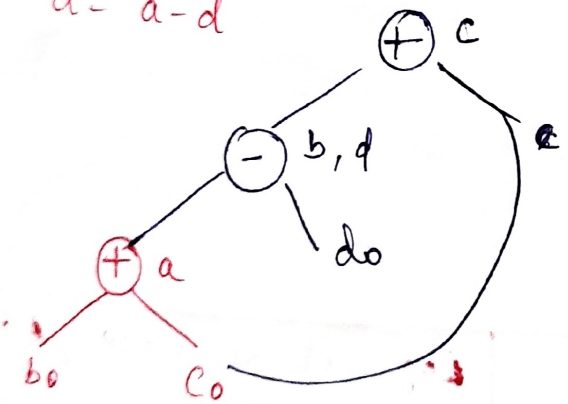
③ Interior nodes are operators.

④ Internal nodes also represent result of the expressions.

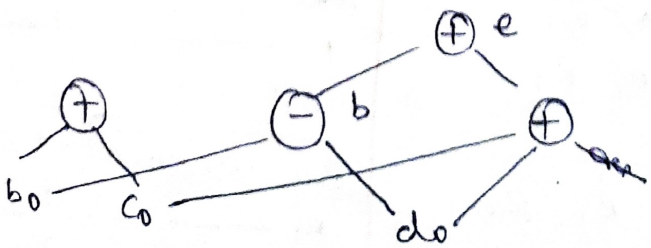
$x = y + z$



Ex 1] $a = b + c$
$b = a - d$
$c = b + c$
$d = a - d$

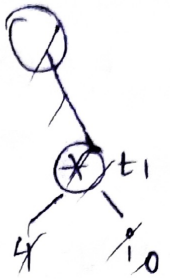


2] $a = b + c$, $b = b - d$, $c = c + d$, $e = b + c$

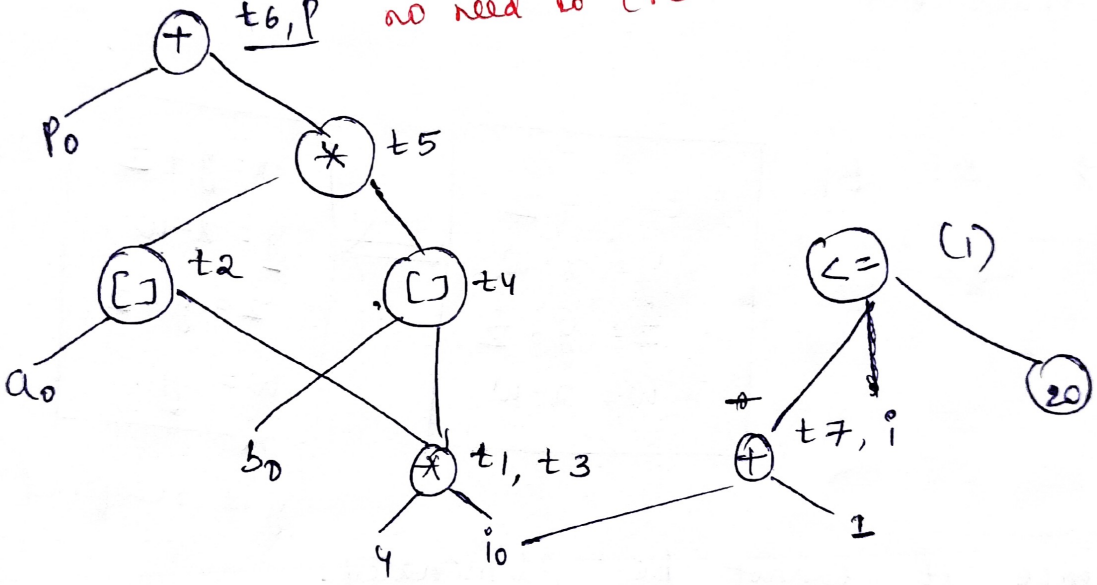


4] 1) $t_1 = 4 * i$
2) $t_2 = a[t_1]$
3) $t_3 = 4 * i$
4) $t_4 = b[t_3]$
5) $t_5 = t_2 * t_4$
6) $t_6 = p + t_5$
7) $p = t_6$
8) $t_7 = i + 1$
9) $i = t_7$
10) if $i <= 20$ goto (1)

p_0 - subscript of should be given to
i_0 -
leaf nodes & should not be given to internal node.



Copy time-



no need to create a new node for

Optimization of Basic Blocks / Transformation of

1. Structure Preserving Transformations
2. Algebraic transformations.

(CSE)

1. Common subExpression Elimination
2. Dead code Elimination
3. Renaming of Temporary variables.
4. Interchange of 2 independent adjacent statements.

→ Ex: B_1

$x = y + z$ $y = x - w$ $z = y + z$ $w = x - w$	$x = y + z$ $y = x - w$ $z = y + z$ $w = y$.

Since x-w is CSE

y is redefined in second

Hence it cannot be eliminated.

(2) Dead Code Elimination:

B_1

$a = a + 2$ $b = b + c$	$b = b + c$

after this a is not used

B_2

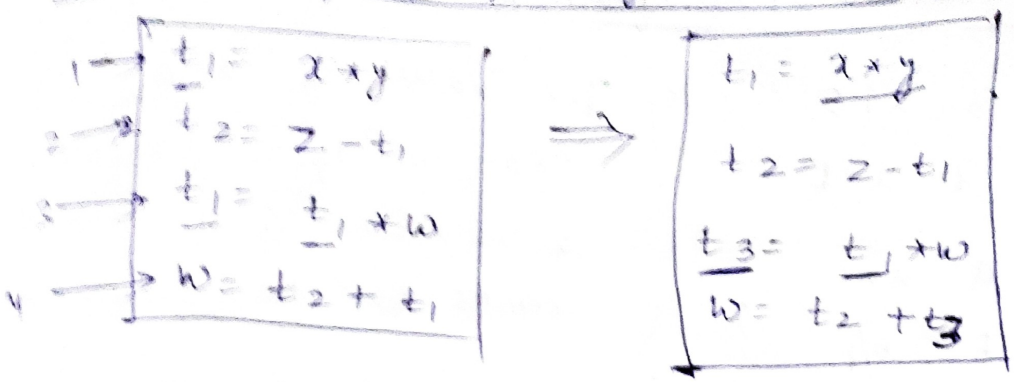
$b = b * c$ $c = b + 2$	$b = b * c$ $c = b + 2$

Assuming that B_2 is the final block, since a is not used later it can be eliminated without altering the remaining lines.

Ex 2) `a = 0;` ⟹ `a = 0;`

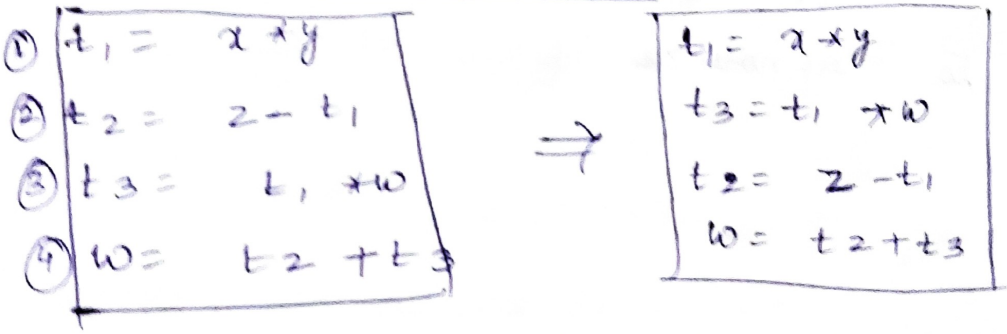
`if (a == 1) {` will never Execute

a) Renaming of temporary variables :



If we want to retain the previously calculated value for future usage, we can rename the variables & the new variable name has to be reflected in all the uses.

b) Interchange of statements



Statements are said to be independent if the variables used in RHS of exp(1) should not be used as LHS of exp2 & vice versa.

Consider stmt ① & ②, t_1 (LHS) of stmt ① is used in RHS of stmt 2.

② & ③ → Independent ← It can be written in any order

③ & ④ → w RHS of stmt ③ is used in LHS of stmt ④

④ & RHS of stmt (3) is used in RHS of stmt ④.

* Algebraic Transformations

$x = x + 0$ ✗
$x = x - 0$

$a = a * 1$
$b = b / 1$ ✗

The value of x, a, & b is not changed after performing the calculations. Then such type of stmts can be eliminated.

$c = d ** 2 \Rightarrow pow(d, 2)$

$\Downarrow$

$c = d * d$

Instead of calling a fun pow() to perform exponent task, It can be easily done by writing $c = d * d$. Such type of algebraic transformations can be done to optimize basic blocks.

Peephole optimization:

Peephole – small moving window on the target machine pgm.

Peephole optimization: Replaces short sequence of target instructions by a shorter or faster sequence. It will consider part by part code & improvises / optimizes the same.

Techniques:

1. Redundant Instruction Elimination
2. Unreachable code ——— " ———
3. Flow of Control optimizations
4. Algebraic simplifications
5. Use of machine idioms.

(1) MOV R0, x → Load instruction.

MOV x, R0 → redundant load & store.

This stmt can be eliminated if this is not preceded by any label. Ex: L1: MOV x, R0.

(2)

```
x=0;
if (x==1)
{
    a=b;
}
```

unnecessary jumps can be removed.

⇒ Int. code

```
x=0
if x=1 goto L1
goto L2
L1: a=b;
L2:
```

```
x=0
if x!=1 goto L2
a=b;
L2:
```

3. Flow of control optimization

① goto L_1
....
L_1: goto L_2
....
L_2: a=b
(2 jumps)

goto L_2
....
L_1: goto L_2
....
L_2: a=b

(unconditional jump)
Ex)
If this stmt is not the target of some other stmt then it can be eliminated

② if a<b goto L_1
....
L_1: goto L_2
⇓
if a<b goto L_2
....

conditional jump Ex.

L_1: goto L_2 → if this stmt is not the target of other stmt then it can be eliminated.

4] Algebraic Simplifications:

x = x+0
x = x*1

} these stmts must be removed since the value of x is not affected after performing calculations.

5] Reduction in strength:

x↑2 = x*x
2*x x+x
x+2 = x≪1
x/2 = x≫1

5. Use of Machine Idioms :

i = i + 1 → INC i
i = i - 1 → DEC i
} → one instruction

requires 3 m/c instructions

Loop Optimization :

Intro to global data flow analysis

→ To do code optimization & code generation

→ Compiler - Collect info about the whole pgm
- distribute it to each block in the flow graph.

Data flow Equation:

$$out[S] = gen[S] \cup (In[S] - KILL[S])$$

out [S] ⟶ Info. at the end of S.

gen [S] → Info generated by S.

in [S] ⟶ Info enters at the beginning of S.

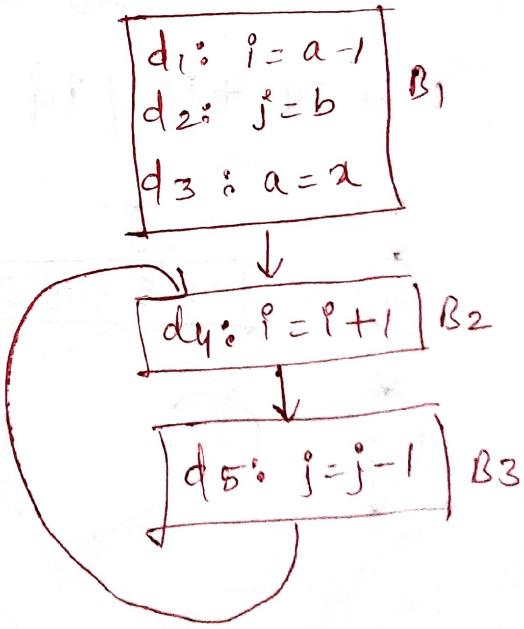
KILL [S] → Info killed by S.

Points & Paths:

Within B_1, there is a point b/w 2 adjacent statements

B_1 - 4 pts

B_2 - 2 pts.



Path:

A path from p_1 to p_n is a sequence of points $p_1, p_2 \cdots p_n$ such that for each i b/w 1 & n-1.

i) P_i - point immediately preceding a stmt.

ii) P_{i+1} - point from following that stmt in same block.

ii) P_i = End of some block

P_{i+1} = beginning of a successor block

Ex; path from ~~beginning~~ End of B_3 to beginning of B_2.

Reaching Definitions

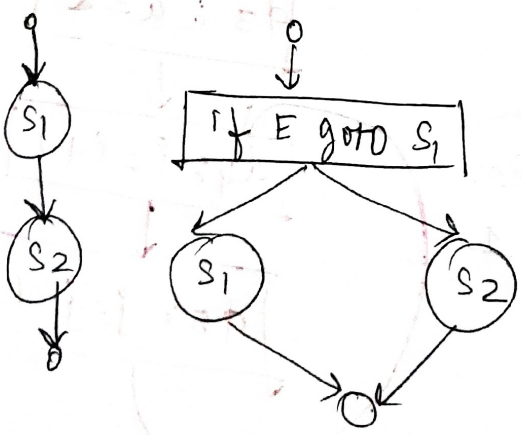
* A def^n of variable x is a statement that assigns a value of x.
* A def^n of d reaches a point p, if there is a path from d to p, such that d is not killed, along the path.

Ex:
```
a = 3
b = a + 2
a = x + y
c = a + 2.
```

Data flow analysis of structured programs:

$S \rightarrow id := E \mid S;S \mid$ if E then S else $S \mid$ do S while E

$E \rightarrow id + id \mid id$

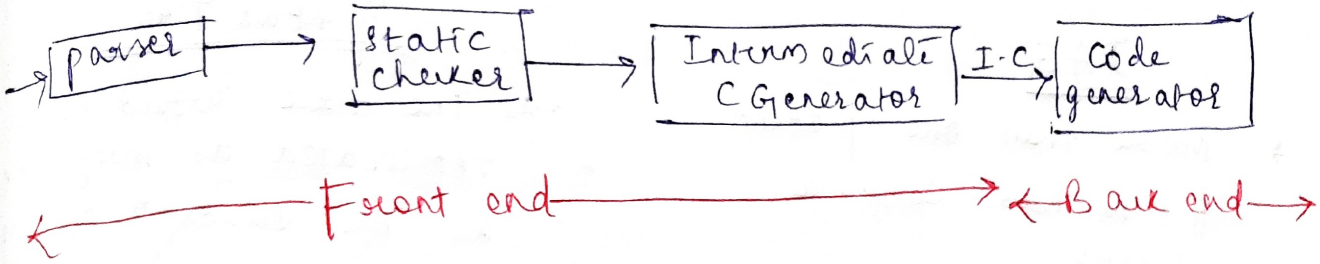


Unit 4

21CS63

Variants of Syntax Tree

Intermediate code generation is the fourth phase of compiler design.



In analysis-synthesis model of a compiler, the front-end analyzes a source pgm & creates an int rep^n & from which backend generates target code.

The sequence of int. rep^{n}s are:

Source pgm → High level I-R → Low level I-R → Target code

* Syntax Trees – High level
* Register allocation & Instruction selection – low level
* Three address code – High level to low level;

Variants of Syntax Trees (ST) → It consists of

* Nodes – constructs
* Children – meaningful components.

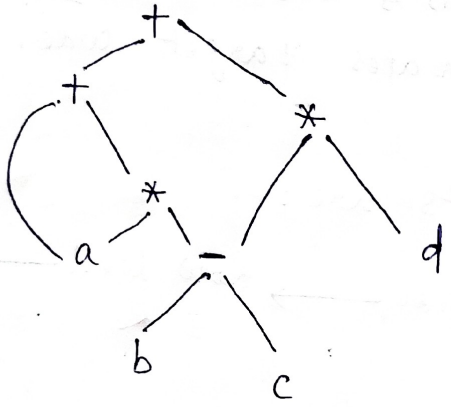
① DAG : DAG for a expression identifies common subexpressions.

Ex : Leaves → operands

Interior nodes → operators.

DAG	Syntax Tree
* More than one parent if N has common subexp^n.	* The tree would be replicated as many times as the subexp^n appears.

Ex : DAG: a + a * (b - c) + (b - c) * d



Intermediate Code Generation

- Linear Form
  - ① Postfix Notation
  - ② 3AC
- Tree Form
  - ③ Syntax Tree
  - ④ DAG

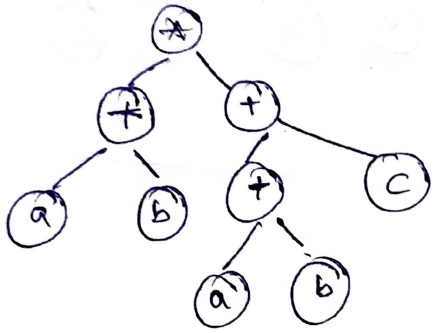
Adv:
Less space
Disadv:
Instruction

Ex:1) $(a+b) * (a+b+c)$

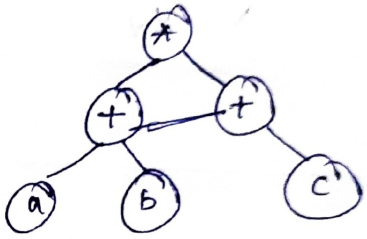
① Postfix notation: $ab+ab+c+*$

② 3AC:
$t_1 = a+b$
$t_2 = a+b$
$t_3 = t_2+c$
$t_4 = t_1 * t_3$

③ Syntax Tree:



④ DAG: nodes should be unique



② $-(a*b)+(c*d+e)$

3AC:
$t_1 = a*b$
$t_2 = -t_1$
$t_3 = c*d$
$t_4 = t_3+e$
$t_5 = t_2+t_4$

Quadruple:

	operator	op1	op2	result
0	*	a	b	t_1
1	-	t_1		t_2
2	*	c	d	t_3
3	+	t_3	e	t_4
4	+	t_2	t_4	t_5

Disadv: Takes extra space

Adv: can move instructions don't affect evaluation

Triples:

	operator	op1	op2
0	*	a	b
1	-	(0)	
2	*	c	d
3	+	(2)	e
4	+	(1)	(3)

Indirect triples:

100	(0)
101	(1)
102	(2)
103	(3)
104	(4)

DA: 2 memory references are required

A: Instructions can be moved.

① $X = (((a+a) + (a+a)) + ((a+a) + (a+a)))$

t_1 t_2 t_4 t_5

3AC

$t_1 = a+a$

$t_2 = a+a$

$t_3 = t_1 + t_2$

$t_4 = a+a$

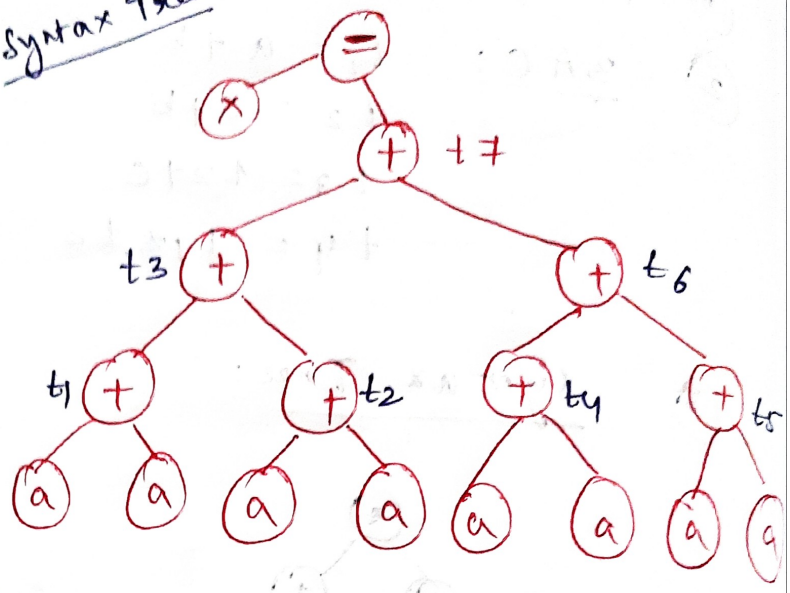
$t_5 = a+a$

$t_6 = t_4 + t_5$

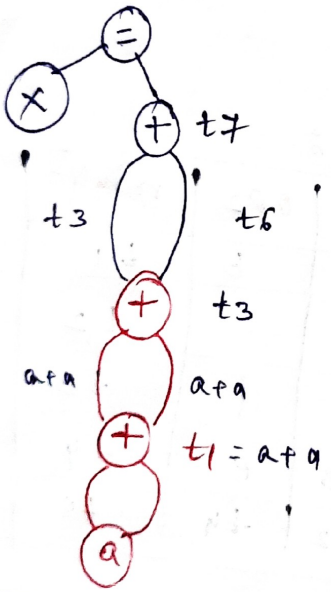
$t_7 = t_3 + t_6$

$X = t_7$

Syntax Tree

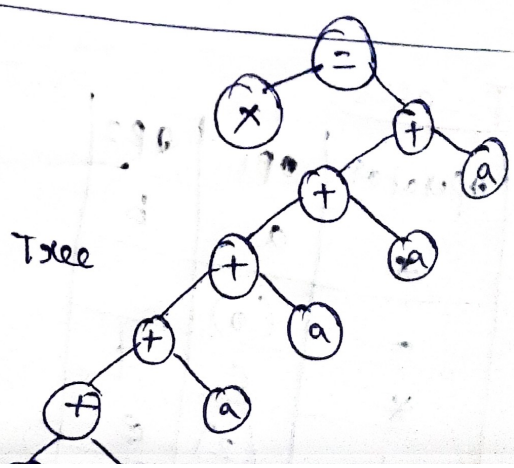


DAG

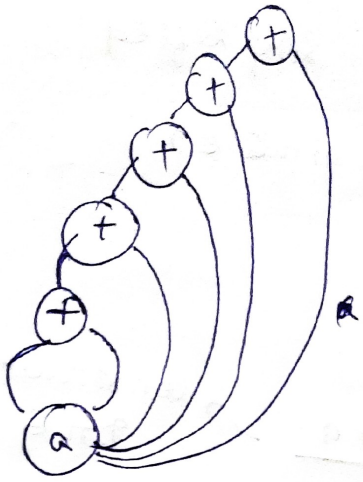


② $X = a + a + a + a + a + a$

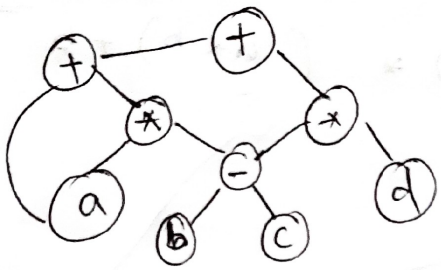
Syntax Tree



DAG :

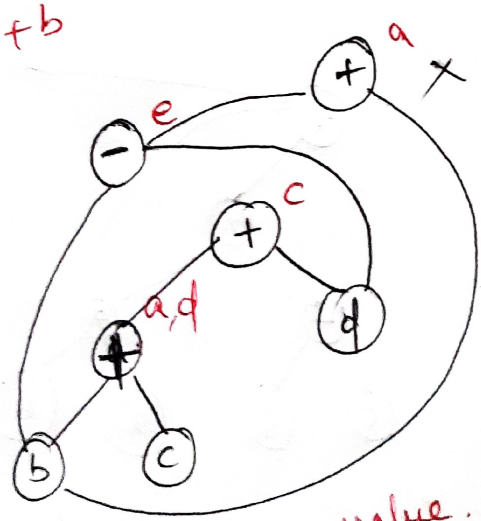


(3) $a + a * (b - c) + (b - c) * d$



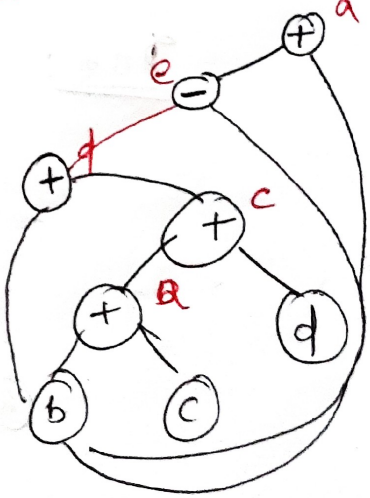
(4). $a = b + c$
$c = a + d$
$d = b + c$
$e = d - b$
$a = e + b$

What is the
→ min no of nodes & edges.



Used first c value.

Always use last modified value



8 nodes
10 edges

Still it can be simplified

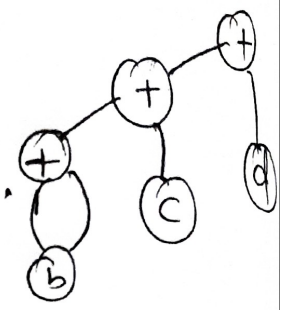
$a = e + b$ (substitute e)

$a = d - b + b$

$a = b + c$

$a \quad b + a + d$

$a = b + b + c + d$ → construct DAG



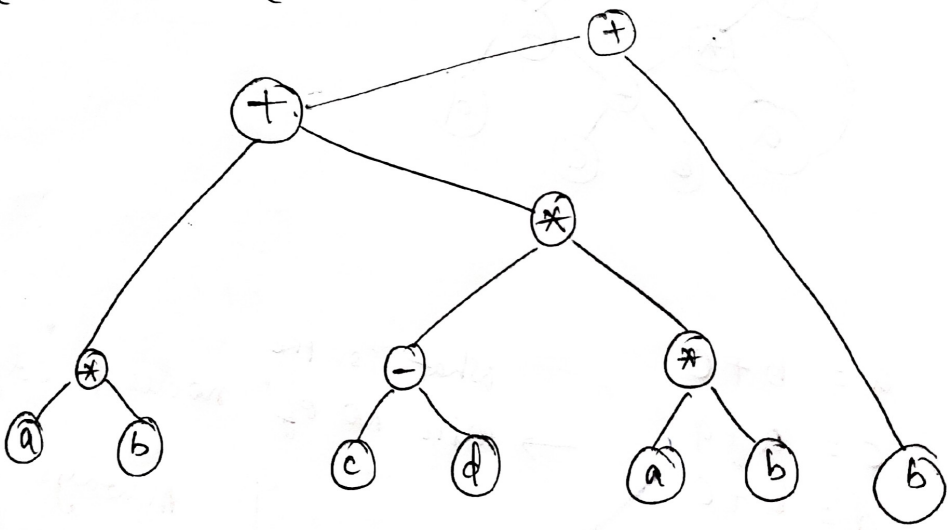
With

6 nodes &
6 edges

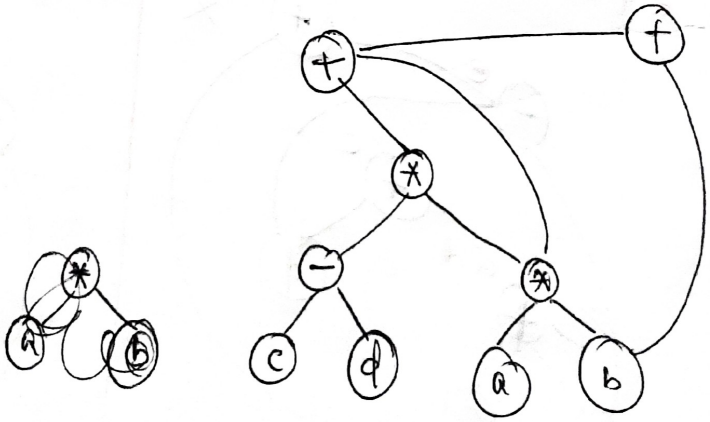
Syntax Tree:

Associativity: L to R & Highest precedence ()

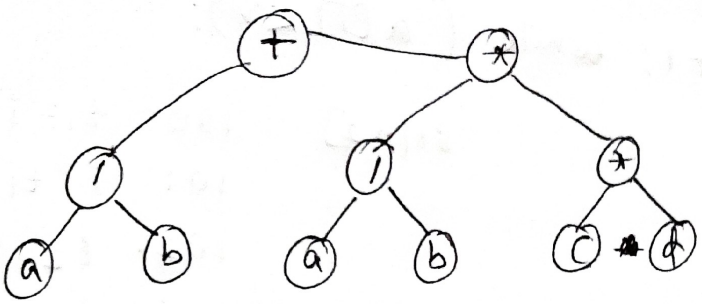
① $(a * b) + (c - d) * (a * b) + b$



DAG:

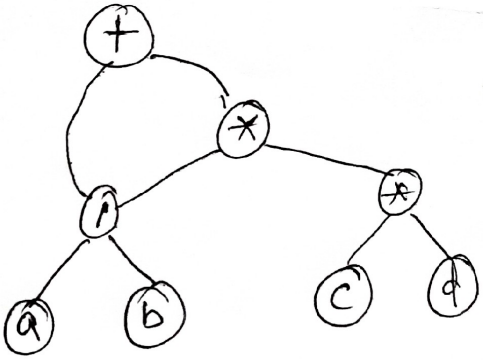


(a) $(a/b) + (a/b) * (c * d)$



→ Syntax Tree

DAG



3 AC

a) do i = i + 1; while (a[i] < v);

s[i]

L: $t_1 = i + 1$
$i = t_1$
$t_2 = i * 8$
$t_3 = a[t_2]$
if $t_3 < v$ goto L

Symbolic Labels

s[n2]

100: $t_1 = i + 1$
101: $i = t_1$
102: $t_2 = i * 8$
103: $t_3 = a[t_2]$
104: if $t_3 < v$ goto

Position numbers